

Student Details

Name: Himanshu

Roll Number: 23F3000651

Student Email: 23f3000651@study.iitm.ac.in

I am currently pursuing the **Diploma level in the BS Degree in Data Science and Applications from IIT Madras**. I am interested in **backend development using Flask, database management, and building full-stack applications**. This project helped me gain practical experience in designing APIs, structuring a database, and integrating a frontend with a backend service.

Project Details

Project Title: Online Placement Management System

Problem Statement:

To design and develop a web-based Campus Placement Management Portal that allows students to view available placement drives, apply for companies, and track their application status. The system also enables administrators to manage company drives and monitor student applications efficiently.

Approach:

The application was built using Flask as the backend framework with a structured modular architecture. The system allows companies to create placement drives, students to browse and apply for drives, and administrators to manage and update application statuses. The platform uses REST APIs for communication between the frontend and backend, while SQLAlchemy ORM is used to manage the database and store information related to students, companies, and applications.

AI / LLM Usage

Some AI tools such as **ChatGPT** were occasionally used for **general guidance, understanding documentation, and resolving minor programming issues** during development around 5-10%. The **overall design, implementation, and testing of the application were completed independently by the myself**.

Technologies and Frameworks Used

Flask

Flask was used as the **backend web framework** to build the application and manage HTTP requests, routing, and server-side logic.

Flask-SQLAlchemy

Flask-SQLAlchemy was used as an **Object Relational Mapper (ORM)** to interact with the database using Python objects instead of writing raw SQL queries.

SQLite

SQLite was used as the **database system** to store information related to students, companies, placement drives, and applications.

Vue.js

Vue.js was used for the **frontend interface** to create dynamic and interactive user components such as dashboards and forms.

Axios

Axios was used to **send HTTP requests from the frontend to backend REST APIs** and retrieve data asynchronously.

HTML, CSS, and Bootstrap

These technologies were used to design the **user interface and responsive layout** of the application.

Database Schema / ER Diagram

Tables:

User — stores authentication and profile details of all users

(id, email, name, password_hash, role, is_active, created_at).

Company — stores company profile information linked to a user account

(id, user_id, company_name, hr_contact, industry, location, website, approval_status).

Student — stores student profile details such as education, skills, experience, and resume link

(id, user_id, student_name, education, skills, experience, resume_link).

PlacementDrive — stores job postings created by companies including job title, job description, eligibility criteria, application deadline, and active status.

Application — stores records of student applications to placement drives

(id, student_id, drive_id, status) with a unique constraint to prevent duplicate applications.

Interview — stores interview scheduling details including interview date, time, status, feedback, and offer letter.

Relationships:

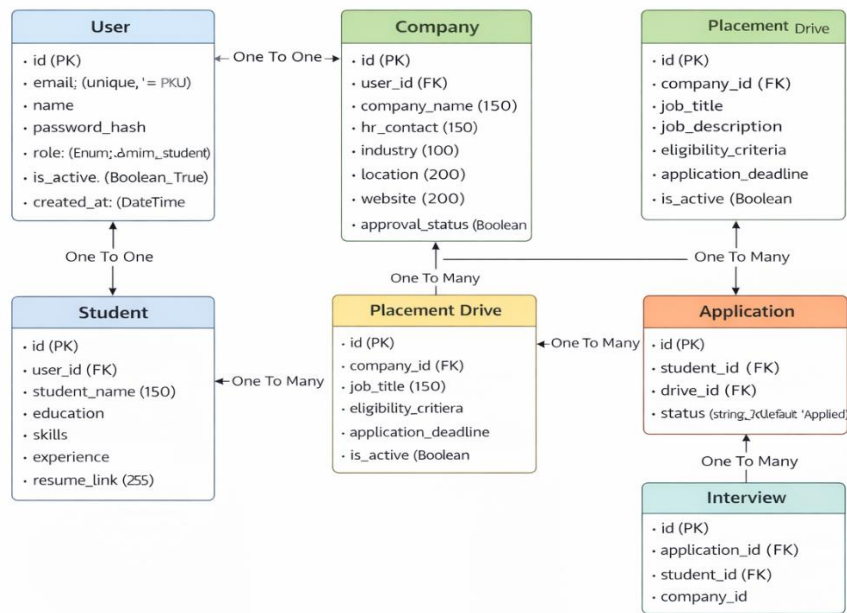
One-to-One → User → Company / Student

One-to-Many → Company → PlacementDrive

One-to-Many → Student → Application

One-to-Many → PlacementDrive → Application

One-to-Many → Application → Interview



API Design

REST APIs were implemented using **Flask routes** to allow communication between the frontend and backend of the application. The APIs handle operations such as retrieving placement drives, submitting student applications, managing company profiles, and scheduling interviews. Standard HTTP methods like **GET, POST, PUT, and DELETE** are used for different operations.

Architecture and Features

Architecture

The project follows a **modular Flask architecture** where database models are defined using **Flask-SQLAlchemy**, route handlers act as controllers for API logic, and the frontend interface is built using templates and Vue components. The backend manages authentication, data storage, and API responses, while the frontend communicates with the backend using HTTP requests.

Features

The system allows **students to create profiles, view placement drives, and apply for jobs**. Companies can **create and manage placement drives**, while administrators can **approve companies and monitor activities**. The platform also includes features such as **application tracking, interview scheduling, and status updates**, providing a complete workflow for managing campus placements.

Video:

<https://drive.google.com/file/d/1oEnYLJR3RVoKTyG5kLvxFjfa4nQTg2uJ/view?usp=sharing>